Applied Machine Learning

Gradient Descent — Fully Worked Solution

The Question

A campus food-delivery startup predicts delivery time from the distance travelled using the model $\hat{y} = wx$, where x is distance in km and $\hat{y}$ is predicted time in minutes. Four completed deliveries are recorded:

$$\mathbf{x\ (km):\ 1,\ 2,\ 3,\ 4} \qquad \mathbf{y\ (min):\ 2,\ 4,\ 6,\ 8}$$

The loss is Mean Squared Error, the initial weight is $w_0 = 0$ and the learning rate is $\eta = 0.01$.

Perform one epoch of training using (a) Batch Gradient Descent, (b) Mini-Batch Gradient Descent with batch size 2, and (c) Stochastic Gradient Descent. Compare the results and comment.

Step 1 — Write down everything that is given

The first thing to write on the answer sheet is the data, in symbols. This is not a formality — it fixes N, and N is what decides how many updates each method will need.

```
GIVEN
  Model:              ŷ = w × x        (no bias term b)
  Number of samples: N = 4
  Data:               (x₁, y₁) = (1, 2)
                      (x₂, y₂) = (2, 4)
                      (x₃, y₃) = (3, 6)
                      (x₄, y₄) = (4, 8)
  Loss:               L = (1/m) × Σ (ŷ − y)²      [Mean Squared Error]
  Initial weight:     w₀ = 0
  Learning rate:      η = 0.01
  Required:           value of w after ONE epoch, by three methods
```

Why this matters: there is only one unknown in this problem, namely w. Everything that follows is a procedure for improving that single number. If you keep that in mind, the three methods stop looking like three formulas and start looking like three schedules for doing the same thing.

Step 2 — Decide what “one epoch” means for each method

Do this before any arithmetic. One epoch means one complete pass through all 4 samples. What changes between the methods is how often you stop and update w during that pass.

Method	m (samples used per gradient)	Number of updates in one epoch
Batch GD	m = N = 4	4 / 4 = 1 update
Mini-Batch GD (batch size 2)	m = 2	4 / 2 = 2 updates
Stochastic GD	m = 1	4 / 1 = 4 updates

Why this step exists: it tells you in advance how many blocks of working to write. If your Batch GD answer has four updates in it, you already know something has gone wrong, without having to recheck a single multiplication.

Step 3 — Derive the gradient of the loss

You are asked to move w in the direction that reduces the loss. To know that direction, differentiate the loss with respect to w. Write the derivation out; it takes four lines.

Line 1 — Write the loss with w visible in it.

Substitute $\hat{y} = wx$ into the loss so that w actually appears, otherwise there is nothing to differentiate:

$$L = (1/m) \times \Sigma (\hat{y} - y)^2 = (1/m) \times \Sigma (wx - y)^2$$

Line 2 — Differentiate the squared term using the chain rule.

The outer function is (something)², whose derivative is 2 × (something). The inner function is (wx − y), whose derivative with respect to w is x, because y is a constant and w is multiplied by x:

$$d/dw\ (wx - y)^2 = 2(wx - y) \times x$$

Line 3 — Put it back inside the summation.

$$\partial L/\partial w = (1/m) \times \Sigma\, 2(wx - y) \times x = (2/m) \times \Sigma (wx - y) \times x$$

Line 4 — Rewrite it in the form you will actually compute with.

Since (wx − y) is exactly ($\hat{y}$ − y), which is the prediction error:

$$\partial L/\partial w = (2/m) \times \Sigma (error) \times x \qquad where\ \ error = \hat{y} - y$$

Two things worth understanding rather than memorising

- **Why the x is there.** It comes from the chain rule. Physically it means a sample with a large x has a larger say in how w should change — which is correct, because w is multiplied by x, so an error on a long delivery blames w more than the same error on a short one.
- **Why the 2 is there, and how to get rid of it.** It comes from differentiating the square. Many textbooks define the loss as $L = (1/2m)\ \Sigma(\hat{y} - y)^2$ purely so that this 2 cancels. Both conventions are correct. Choose one, write it down in your first line, and do not switch.

Step 4 — Write the update rule

The gradient points in the direction where the loss increases fastest. You want the loss to decrease, so you move the opposite way. That is the entire meaning of the word descent:

$$w \leftarrow w - \eta \times (\partial L/\partial w) \qquad i.e.\ \ new\, w = old\, w - 0.01 \times gradient$$

One consequence to note now, because it saves confusion later: every gradient in this problem will come out negative (the model under-predicts at the start, so all errors are negative). Subtracting a negative number means w will always increase. So if at any point your w decreases, you have made a sign error.

> **This same rule is used by all three methods. Nothing about the formula changes. Only m changes — that is, how many samples you look at before applying it.**

Step 5 — Method 1: Batch Gradient Descent

What I do: compute the error for all 4 samples first, add up the contributions, and only then update w one single time.

5.1 Compute the prediction and error for each sample

The current weight is w = 0, so every prediction is 0 × x = 0. Work through the samples one at a time:

```
Sample 1  (x = 1, y = 2)   with w = 0
  ŷ = w × x       = 0 × 1  = 0
  error = ŷ − y   = 0 − 2  = −2
  error × x       = −2 × 1 = −2

Sample 2  (x = 2, y = 4)   with w = 0
  ŷ = w × x       = 0 × 2  = 0
  error = ŷ − y   = 0 − 4  = −4
  error × x       = −4 × 2 = −8

Sample 3  (x = 3, y = 6)   with w = 0
  ŷ = w × x       = 0 × 3  = 0
  error = ŷ − y   = 0 − 6  = −6
  error × x       = −6 × 3 = −18

Sample 4  (x = 4, y = 8)   with w = 0
  ŷ = w × x       = 0 × 4  = 0
  error = ŷ − y   = 0 − 8  = −8
  error × x       = −8 × 4 = −32
```

Collected into a table, which is what you should actually draw in the answer sheet:

Sample	x	y	$\hat{y} = wx$	error = $\hat{y} - y$	error × x
1	1	2	0	−2	−2
2	2	4	0	−4	−8
3	3	6	0	−6	−18
4	4	8	0	−8	−32
				Σ (error × x) =	**−60**

5.2 Add up the contributions

```
Σ (error × x) = (−2) + (−8) + (−18) + (−32)
              = −10 + (−18) + (−32)
              = −28 + (−32)
              = −60
```

5.3 Compute the gradient

Here m = 4, because all four samples were used to form this one gradient:

```
∂L/∂w = (2/m) × Σ (error × x)
      = (2/4) × (−60)
      = 0.5 × (−60)
      = −30
```

5.4 Apply the update rule — once

```
w = w − η × (∂L/∂w)
  = 0 − 0.01 × (−30)
  = 0 + 0.30
  = 0.30
```

Batch GD, after 1 epoch: w = 0.30 (1 update performed)

Understand what just happened: the entire dataset was consumed to produce a single number, −30, and that number moved w exactly once. This gradient is the true gradient of the loss over the

whole dataset — it is as accurate as it is possible to be. But one pass through the data bought only one step.

Step 6 — Method 2: Mini-Batch Gradient Descent (batch size = 2)

What I do: split the 4 samples into two groups of 2. Process the first group and update w. Then process the second group **using the new value of w**, and update again. Two updates in the epoch.

Here m = 2, so the gradient factor is 2/m = 2/2 = 1. The factor has not been dropped — for this particular batch size it simply happens to equal one.

6.1 Update 1 — Mini-batch A = {(1, 2), (2, 4)}, starting with w = 0

```
Sample 1  (x = 1, y = 2)   with w = 0
  ŷ = 0 × 1  = 0
  error     = 0 − 2  = −2
  error × x = −2 × 1 = −2

Sample 2  (x = 2, y = 4)   with w = 0
  ŷ = 0 × 2  = 0
  error     = 0 − 4  = −4
  error × x = −4 × 2 = −8

Σ (error × x) = (−2) + (−8) = −10

∂L/∂w = (2/2) × (−10) = 1 × (−10) = −10

w = 0 − 0.01 × (−10)
  = 0 + 0.10
  = 0.10
```

After the first mini-batch, w = 0.10. This value must now be carried into the next mini-batch.

6.2 Update 2 — Mini-batch B = {(3, 6), (4, 8)}, starting with w = 0.10

Note carefully: the predictions are no longer zero, because w is no longer zero. This is the single most common place where marks are lost.

```
Sample 3  (x = 3, y = 6)   with w = 0.10
  ŷ = 0.10 × 3  = 0.30
  error     = 0.30 − 6   = −5.70
  error × x = −5.70 × 3 = −17.10

Sample 4  (x = 4, y = 8)   with w = 0.10
  ŷ = 0.10 × 4  = 0.40
  error     = 0.40 − 8   = −7.60
  error × x = −7.60 × 4 = −30.40

Σ (error × x) = (−17.10) + (−30.40) = −47.50

∂L/∂w = (2/2) × (−47.50) = −47.50

w = 0.10 − 0.01 × (−47.50)
  = 0.10 + 0.475
  = 0.575
```

Mini-Batch GD, after 1 epoch: w = 0.575 (2 updates performed)

Understand what just happened: the second mini-batch was evaluated with an already-improved w, so its predictions were better and its errors smaller than they would have been at w = 0. Even so, it still moved w a further 0.475. Two updates instead of one carried the epoch further than Batch GD managed, from exactly the same four samples.

Step 7 — Method 3: Stochastic Gradient Descent

What I do: take one sample, update w immediately, and move to the next sample using the updated w. Four samples means four updates.

Here m = 1, so the gradient formula collapses to a single term — there is no summation left to perform:

$$\partial L/\partial w \;=\; (2/1) \times (error) \times x \;=\; 2 \times (error) \times x$$

7.1 Update 1 — Sample (x = 1, y = 2), starting with w = 0

```
ŷ        = 0 × 1 = 0
error    = 0 − 2 = −2
gradient = 2 × (−2) × 1 = −4
w        = 0 − 0.01 × (−4)
         = 0 + 0.04
         = 0.04
```

7.2 Update 2 — Sample (x = 2, y = 4), starting with w = 0.04

```
ŷ        = 0.04 × 2 = 0.08
error    = 0.08 − 4 = −3.92
gradient = 2 × (−3.92) × 2 = −15.68
w        = 0.04 − 0.01 × (−15.68)
         = 0.04 + 0.1568
         = 0.1968
```

7.3 Update 3 — Sample (x = 3, y = 6), starting with w = 0.1968

```
ŷ        = 0.1968 × 3 = 0.5904
error    = 0.5904 − 6 = −5.4096
gradient = 2 × (−5.4096) × 3 = −32.4576
w        = 0.1968 − 0.01 × (−32.4576)
         = 0.1968 + 0.324576
         = 0.521376
```

7.4 Update 4 — Sample (x = 4, y = 8), starting with w = 0.521376

```
ŷ        = 0.521376 × 4 = 2.085504
error    = 2.085504 − 8 = −5.914496
gradient = 2 × (−5.914496) × 4 = −47.315968
w        = 0.521376 − 0.01 × (−47.315968)
         = 0.521376 + 0.47315968
         = 0.99453568   ≈  0.9945
```

Summarised as a table for the answer sheet:

Update	Sample (x, y)	w before	ŷ	error	gradient	w after
1	(1, 2)	0	0	−2	−4	0.0400
2	(2, 4)	0.0400	0.0800	−3.9200	−15.68	0.1968
3	(3, 6)	0.1968	0.5904	−5.4096	−32.46	0.5214
4	(4, 8)	0.5214	2.0855	−5.9145	−47.32	0.9945

Stochastic GD, after 1 epoch: w ≈ 0.9945 (4 updates performed)

A note on rounding: carry the full decimal value from one update into the next and round only at the very end. If you round to two decimals at each step, the rounding errors accumulate through the chain and your final answer will visibly differ from the expected one.

Step 8 — Compute the loss for each result

Comparing the three values of w is more convincing if you also show what each one does to the loss. The loss is computed over all four samples, whichever method produced the w.

8.1 Loss at the start, w = 0

```
ŷ values : 0, 0, 0, 0
errors   : -2, -4, -6, -8
squares  : 4, 16, 36, 64
sum      : 4 + 16 + 36 + 64 = 120
MSE      : 120 / 4 = 30.00
```

8.2 Loss after Batch GD, w = 0.30

```
ŷ values : 0.30, 0.60, 0.90, 1.20
errors   : -1.70, -3.40, -5.10, -6.80
squares  : 2.89, 11.56, 26.01, 46.24
sum      : 2.89 + 11.56 + 26.01 + 46.24 = 86.70
MSE      : 86.70 / 4 = 21.68
```

8.3 Loss after Mini-Batch GD, w = 0.575

```
ŷ values : 0.575, 1.150, 1.725, 2.300
errors   : -1.425, -2.850, -4.275, -5.700
squares  : 2.0306, 8.1225, 18.2756, 32.4900
sum      : 60.9188
MSE      : 60.9188 / 4 = 15.23
```

8.4 Loss after Stochastic GD, w = 0.9945

```
ŷ values : 0.9945, 1.9890, 2.9835, 3.9780
errors   : -1.0055, -2.0110, -3.0165, -4.0220
squares  : 1.0110, 4.0441, 9.0993, 16.1765
sum      : 30.3309
MSE      : 30.3309 / 4 = 7.58
```

Step 9 — Compare and comment

All three methods used the same four samples, exactly once each, with the same learning rate and the same starting point.

Method	Samples per update	Updates in the epoch	w after 1 epoch	MSE after 1 epoch
Start	—	—	0	30.00
Batch GD	4	1	0.3000	21.68
Mini-Batch GD (b = 2)	2	2	0.5750	15.23
Stochastic GD	1	4	0.9945	7.58

True optimum for this data: w = 2, MSE = 0.

Why the three answers differ, when the data was identical

Because the number of updates differed. Batch GD moved w once; SGD moved it four times. Each SGD update started from an already-improved w, so the progress compounded within the same single pass. That is the whole reason SGD is nearest the optimum here.

Why this does not make SGD simply the best

Because the quality of each step differed too. Batch GD's single gradient, −30, was the exact gradient of the loss over the full dataset — a small step, but pointed in precisely the correct direction. Each SGD gradient came from one lone sample, which is only a rough estimate of what the full dataset wants.

In this particular question the estimate happened to be excellent, because the data is perfectly consistent — every sample agrees that y = 2x, so no sample disagrees with any other. **Real data is not like this.** If one delivery had been delayed by rain, that sample's gradient would have pulled w in a wrong direction, and SGD would have followed it immediately with no other sample to average against. This is why SGD is described as noisy: it converges quickly but jitters around the minimum rather than settling into it.

Where mini-batch fits, and why it is used in practice

Mini-batch sits between the two on both counts: several updates per epoch, and each gradient averaged over a small group so that one odd sample cannot dictate a whole step. There is also a computational reason — a mini-batch is a matrix, and a GPU processes a matrix of 64 samples far faster than 64 samples one after another. This combination of statistical stability and hardware efficiency is why essentially all practical training uses mini-batch gradient descent.

Step 10 — Check the answer before moving on

Four checks, each taking a few seconds, that catch nearly every error in this type of numerical:

- **Did w increase every time?** All errors were negative here, so every gradient was negative, so every update added to w. A w that decreased means a sign was flipped somewhere.
- **Did the loss decrease?** 30.00 → 21.68, 15.23, 7.58. If the loss went up, either η is too large or the gradient sign is wrong.
- **Is the number of updates correct?** 1, 2 and 4 respectively for N = 4. If Batch GD has more than one update in your working, you have written mini-batch or SGD by mistake.
- **Did each step use the latest w?** In Step 6.2 the predictions must be 0.30 and 0.40, not 0 and 0. In Step 7 the "w before" column must equal the previous row's "w after".

Mistakes to avoid

- **Reusing $w_0 = 0$ for every update.** The most common error by far. Mini-batch and SGD are sequential — each update begins where the last one ended.
- **Dropping the factor 2.** With $L = (1/m)\Sigma(\hat{y} - y)^2$ the gradient is $(2/m)\Sigma(\text{error})(x)$. If you want the 2 gone, define the loss as $(1/2m)\Sigma(\hat{y} - y)^2$ and say so in the first line.
- **Writing error = y − ŷ.** The derivation gives ŷ − y. Reversing it reverses the sign of every gradient and drives w away from the optimum.
- **Forgetting the × x.** The gradient is error × x, never error alone. The x arrives from the chain rule.
- **Treating one epoch as one update.** An epoch is one pass over the data. How many updates fit inside that pass depends entirely on m.
- **Rounding at every step.** Keep the full decimals through the SGD chain; round only the final answer.